PREP PRESENTATION — IE 643 COURSE PROJECT, AUTUMN 2026

Making Large Models Draw Topological Graphs

Teaching a language model to read quantum-circuit code and draw the circuit's DAG — without running the code.

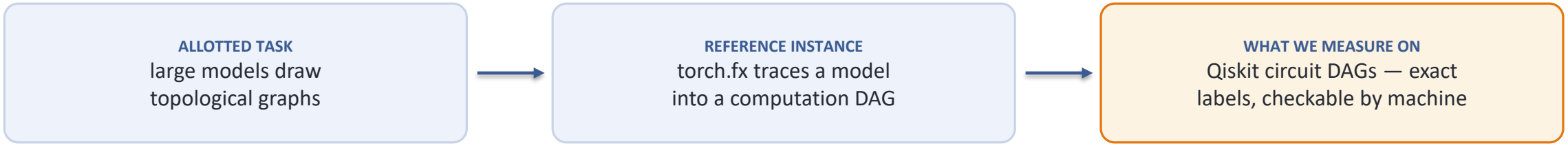
Team Pensieve

Adarsh A Rao · Roll No. 24B2132

Indian Institute of Technology Bombay · September 2026

Outline

- Background: what a quantum circuit is, and why a circuit is a DAG
- The task stated precisely, and why it is hard for a language model
- Background material read, and the three ways to adapt a model
- What we have built: generator, ground truth, validator, metrics
- The dataset: four difficulty tiers, with an example of each
- Where we stand today, the roadmap of subtasks, and the research questions

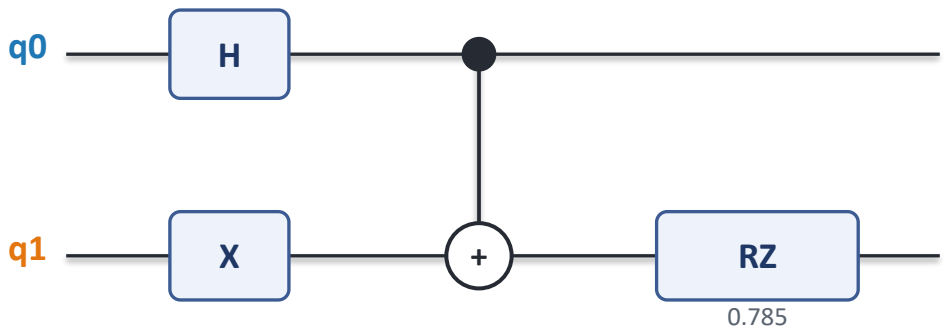


A quantum circuit, in one picture

five lines of Qiskit…

```
qc = QuantumCircuit(2)
qc.h(0)
qc.x(1)
qc.cx(0, 1)
qc.rz(0.785, 1)
```

…build this circuit



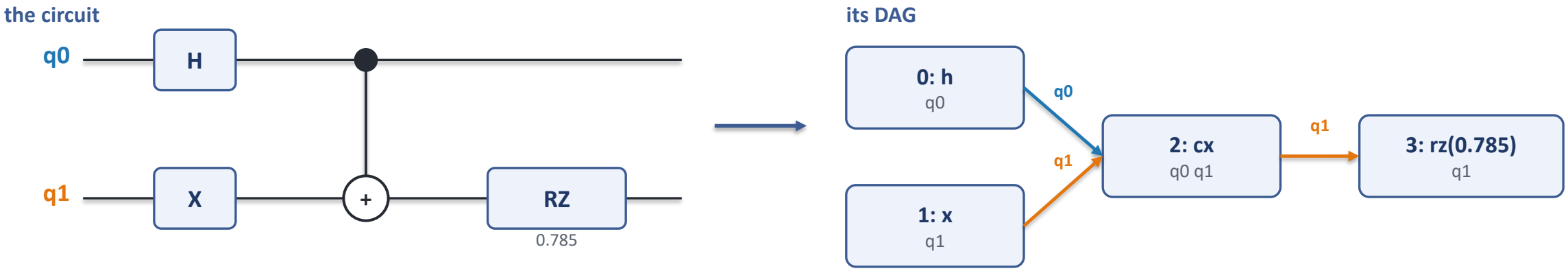
a qubit is a wire

a gate is a box on a wire

cx spans two wires: • control, ⊕ target

straight-line: no branching at run time

The same circuit, as a graph



one node per operation

edge A → B = next operation on that wire

no cycles: time only moves forward

qc.h(0) and qc.x(1) can be written in either order and give the same DAG — so we compare graphs, not gate lists.

Where the correct answer comes from

the entire labelling pipeline

```
from qiskit import QuantumCircuit
from qiskit.converters import circuit_to_dag

qc = QuantumCircuit(2)     # two wires
qc.h(0)                    # append one
qc.x(1)                    # operation
qc.cx(0, 1)                # per call
qc.rz(0.785, 1)

dag = circuit_to_dag(qc)   # DAGCircuit
```

- **QuantumCircuit(n)** → n wires
- **each qc.<gate>(…)** → appends exactly one operation
- **circuit_to_dag** → DAGCircuit, the IR Qiskit's own compiler uses for optimisation and qubit routing
- So: exact labels, zero hand-annotation, unlimited quantity.
- The model is given the source text only.

The task, stated precisely

INPUT — what the model is given

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2)
qc.h(0)
qc.x(1)
qc.cx(0, 1)
qc.rz(0.785, 1)
```



OUTPUT — what it must produce, exactly

```
QUBITS 2
CLBITS 0
NODES
0 h q0
1 x q1
2 cx q0 q1
3 rz(0.785) q1
EDGES
0 -> 2 : q0
1 -> 2 : q1
2 -> 3 : q1
```

node: <id> <gate>[(<params>)] <wires…>

edge: <src> -> <dst> : <wires…>

ids are the model's choice — we compare up to isomorphism

Why this is hard for a language model

- Unroll loops, expand calls, discard dead branches, broadcast qc.h(range(n)).
- Track “last operation on each wire” — with no scratch memory except the tokens already written.
- One missed gate breaks a chain, and the output stops being a valid DAG.
- Our output is almost entirely numbers, and numbers tokenise badly:

`rz(0.785) q12` as the model actually reads it



CRUXEval (2024): models that write code well are substantially worse at predicting what that code does.

BACKGROUND MATERIAL READ

Three ways to use a model — our three phases

Phase 1 · Prompting

DATA
0–5 examples in the prompt

COMPUTE
inference only

PYTHON
known, from pretraining

LIKELY FAILURE
format errors; loses track of wires

Phase 2 · LoRA fine-tuning

DATA
~1k–10k (code, DAG) pairs

COMPUTE
one free GPU, hours

PYTHON
known, from pretraining

LIKELY FAILURE
overfits our generator's style

Phase 3 · From scratch

DATA
10k–1M — we generate them

COMPUTE
one GPU, hours to days

PYTHON
none — learnt from our data

LIKELY FAILURE
poor generalisation to real code

Background material read

FOUNDATIONS — how language models work

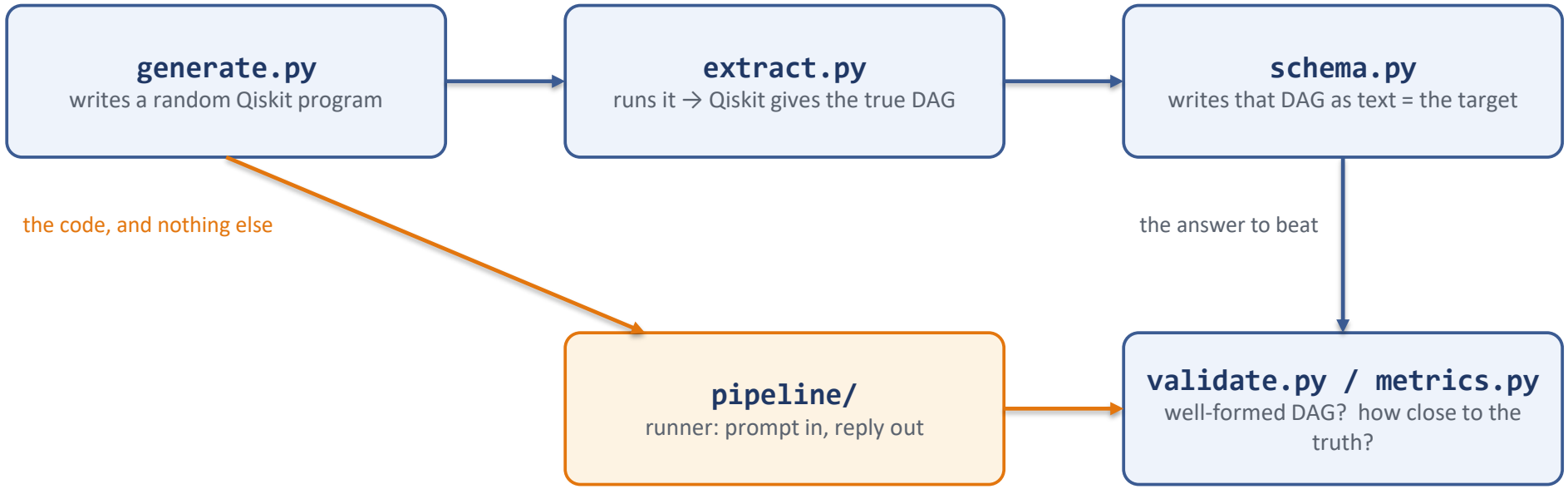
- **3Blue1Brown, Neural Networks** — what attention computes, visually
- **Karpathy, Intro to LLMs** — pretraining → SFT → preference tuning
- **Alammar, The Illustrated Transformer** — the architecture, block by block
  - Taken from these: generation is next-token prediction repeated, so an early mistake poisons the rest; attention is the mechanism that finds the last node on a wire; we decode at temperature 0.

DIRECTLY RELATED WORK

- **Talk like a Graph (2023)** — how a graph is written as text changes LLM accuracy: our output format is a variable to test
- **CRUXEval (2024)** — predicting what code does, without running it: the input half of our task
- **Qiskit HumanEval (2024)** — 151 human-written Qiskit tasks: our source of real code for a held-out test set
- **Qiskit DAGCircuit docs** — defines our target structure exactly
  - No public dataset of (code → circuit DAG) pairs exists. Building one is part of the contribution.

WORK DONE

What has been built so far



seven modules under graphart/

runner + scorer written and dry-run

not yet done: call a real model

The model is handed the code and nothing else — never the circuit, never the answer.

The dataset: four tiers of difficulty

Tier	What the code looks like	Unique samples	Nodes min / avg / max
1 flat	one gate per line — reading it is copying	500	3 / 9.2 / 17
2 loops	for loops over qubits, repeated layers, % n rings, reversed ranges	500	3 / 13.5 / 42
3 control	helper functions, if/else on the loop index, dead branches, nested loops, while, lists of angles	500	3 / 8.6 / 23
4 algorithms	GHZ, QFT, Bernstein–Vazirani, Grover (2q), Deutsch–Jozsa, hardware-efficient ansatz	93	6 / 17.4 / 26

The tiers vary how well the code hides the circuit — not how big the circuit is.

the next four slides: one worked example per tier

tier 4 is 93 samples — test-only until more templates are added

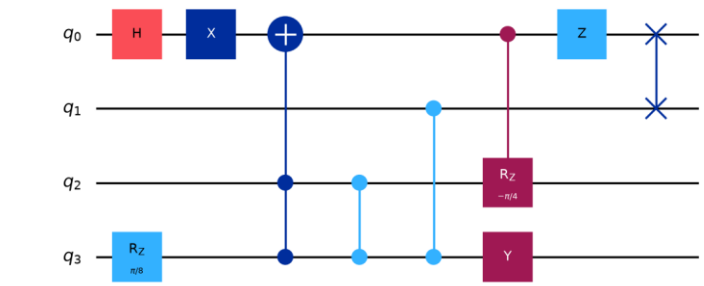
Tier 1 — flat

Tier 1: flat gate list (t1-00246)

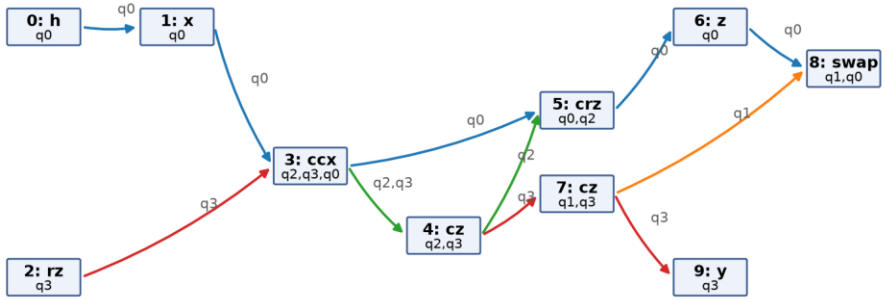
1. source code (model input)

```
qc = QuantumCircuit(4)
qc.h(0)
qc.rz(pi / 8, 3)
qc.x(0)
qc.ccx(2, 3, 0)
qc.cz(2, 3)
qc.cz(1, 3)
qc.crz(-pi / 4, 0, 2)
qc.y(3)
qc.z(0)
qc.swap(1, 0)
```

2. the circuit it builds



3. circuit DAG (model output)



Every operation is written out. Reading the code is transcription; only the edges have to be worked out.

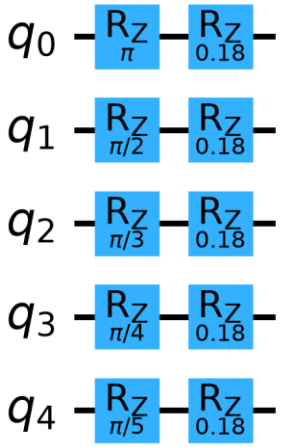
Tier 2 — loops

Tier 2: loops (t2-00099)

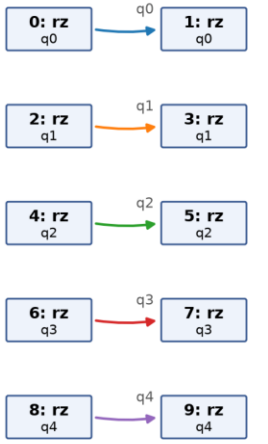
1. source code (model input)

```
num_qubits = 5
qc = QuantumCircuit(num_qubits)
for i in range(num_qubits):
    qc.rz(pi / (i + 1), i)
for i in range(num_qubits):
    qc.rz(0.18, i)
```

2. the circuit it builds



3. circuit DAG (model output)



Two loops over five qubits. Ten operations, five independent chains — and not one of them appears in the code.

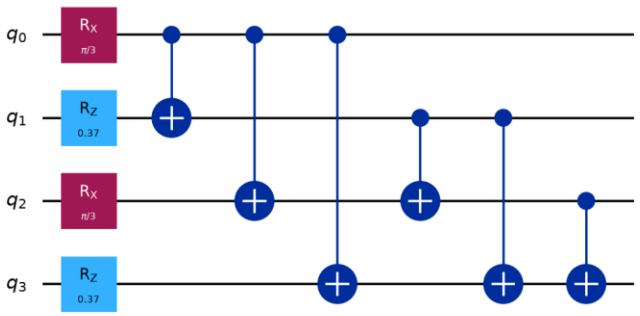
Tier 3 — control flow

Tier 3: functions / if-else / nested loops (t3-00483)

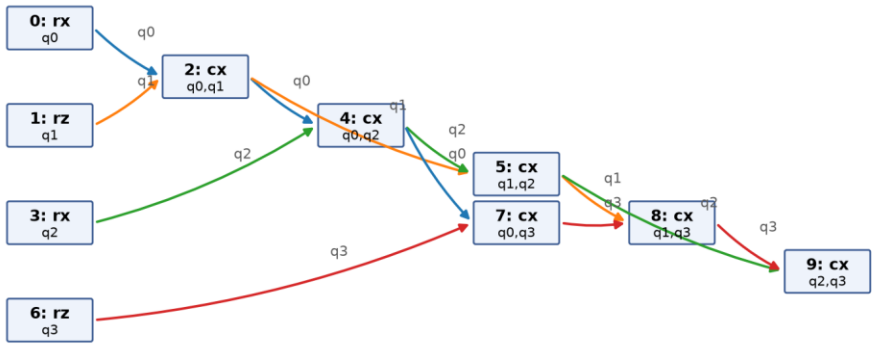
1. source code (model input)

```
n = 4
qc = QuantumCircuit(n)
for i in range(n):
    if i % 2 == 0:
        qc.rx(pi / 3, i)
    else:
        qc.rz(0.37, i)
for i in range(n):
    for j in range(i + 1, n):
        qc.cx(i, j)
```

2. the circuit it builds



3. circuit DAG (model output)



if/else on the loop index picks RX or RZ; a nested loop produces six cx gates. Ten operations, none written down.

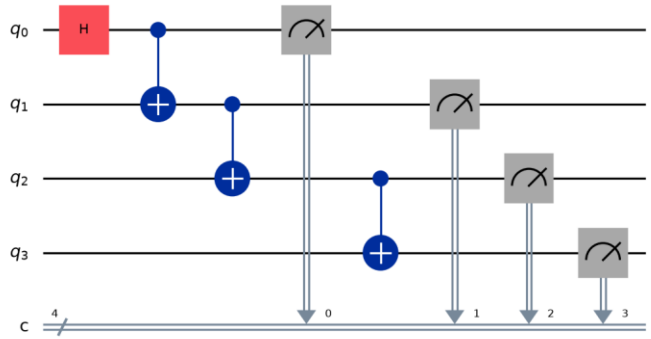
Tier 4 — textbook algorithms

Tier 4: textbook algorithm (t4-00005)

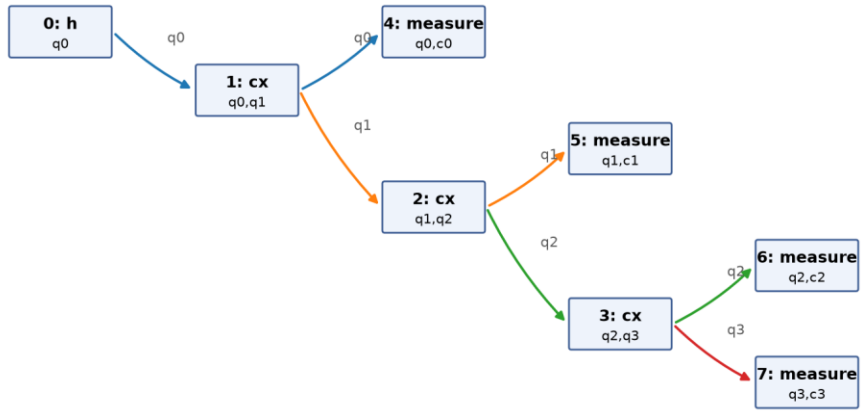
1. source code (model input)

```
n = 4
qc = QuantumCircuit(n, n)
qc.h(0)
for i in range(n - 1):
    qc.cx(i, i + 1)
qc.measure(range(n), range(n))
```

2. the circuit it builds



3. circuit DAG (model output)



A GHZ state with measurement. Classical bits become wires too, so measure nodes carry both a qubit and a clbit.

Data curation: how the splits are kept honest

- **One command, fixed seed.** Nothing is hand-labelled or hand-corrected.
- **Every ground-truth graph passes our own validator before it is kept.** A malformed extraction fails the build instead of poisoning the data.
- **Deduplicated by the graph itself, across the whole dataset.** A sample whose DAG has already been seen is thrown away — so no graph can appear in two splits, and memorising cannot score.
- **Filtered to 3–60 nodes,** which keeps every sample inside a small model's context window.
- **Then split 80 / 10 / 10 within each tier:** 1274 train · 159 val · 160 test. The generator scales to 4,000 per tier for tiers 1–3 (≈ 9,700 training pairs) when fine-tuning needs more.

THE DATASET

Real code for the generalisation tests

Source	What it is	How we would use it
Qiskit HumanEval	151 hand-written Qiskit tasks with reference solutions	run each solution that builds a circuit, extract its DAG → a real-code test set
MQT Bench	thousands of benchmark circuits, 2–130 qubits, generated in Python	more tier-4 algorithms, and larger circuits for the size-generalisation test
QASMBench	~130 OpenQASM circuits from chemistry, arithmetic and ML	load the QASM into a QuantumCircuit, then the same extractor
QCircuitBench	a large dataset for LLM-based quantum algorithm design	algorithm code written in a style quite different from our generator

All four end up as a QuantumCircuit, so the same circuit_to_dag gives the ground truth and the same metrics score it.

our generator has a house style — a fixed vocabulary of gates and code shapes

a model could learn that style instead of the rule; only real code exposes it

EVALUATION

How an answer is scored

Metric	What it means
valid	the output parses; there are no cycles; every edge's wires belong to both of its endpoints; and each wire forms one single chain
exact	the same graph as the ground truth, by labelled graph isomorphism — this is the headline number
node F1	overlap of the (gate, qubits, clbits, params) multisets — partial credit for getting most operations right
edge F1	overlap of the (source label, target label, wires) multisets — partial credit for the structure

sanity check — scoring a correct graph and two damaged copies

```
perfect copy      valid=True   exact=True    node_f1=1.00  edge_f1=1.00
one wrong gate    valid=True   exact=False   node_f1=0.93  edge_f1=0.92
one missing edge  valid=False  exact=False   node_f1=1.00  edge_f1=0.96   ['wire q2: 5 nodes should be chained by 4 edges, found 3']
```

- Reported per tier and per circuit size. Node ids are ignored throughout.

Where we stand today

- **The harness runs end to end, and is checked.** The sanity check on the previous slide is the evidence.
- **The dataset is built and inspected.** 1,593 samples across four tiers, with a rendered example per tier.
- **The Phase-1 harness is written, and dry-run end to end.** Four prompt variants (zero-shot, few-shot, chain-of-thought, and the gate-list ablation), a runner with a hosted and a local backend, and a scorer that reports by tier and by circuit size.
- **No model has been called yet.** We would rather report no numbers than numbers we cannot stand behind. The next command run is the zero-shot baseline over the 160-sample test split.
- **First measurements:** zero-shot and few-shot valid / exact / node F1 / edge F1 on the 160-sample test split, by tier and by circuit size, for one hosted model and one small local model.
- **Then the ablation that answers the central question:** the model outputs only the unrolled gate list, and graphart/connect.py builds the edges. That edge-builder already reproduces the ground truth on all 160 test samples, so whatever it then scores is purely the model's reading of the code.

Roadmap: the project as subtasks

- **Phase 1 — prompting baselines (the next two weeks).** Zero-shot, few-shot, chain-of-thought, and the "LLM unrolls, code connects" ablation, on one hosted model and one local model.
- **Phase 2 — LoRA fine-tuning.** A 0.5B–1.5B open code model on a free Colab or Kaggle GPU. Does a fine-tuned 1B model beat a prompted frontier model? How much data is enough?
- **Phase 3 (stretch) — a tiny transformer from scratch.** nanoGPT-scale, 10–50M parameters, possibly with a custom tokenizer in which q12 is one token.
- **Phase 4 — generalisation.** Train on ≤ 5 qubits, test on 6–10; hold out a construct such as while loops; test on real Qiskit HumanEval code; and revisit torch.fx graphs as a second domain.
- **Phase 5 — interface.** Paste code, see the predicted DAG beside the ground truth with the differences highlighted.
- **Immediate subtasks:** (1) run the zero-shot baseline, (2) freeze the test split, (3) the Qiskit HumanEval importer, (4) more tier-4 templates.

The research questions

- **1. How does accuracy fall as the code hides more of the structure?** Tier 1 → 4 at constant circuit size, and separately as the circuit grows.
- **2. Is the hard part executing the code, or building the graph?** The Phase-1 ablation answers this, and it decides where the rest of the effort goes.
- **3. Prompting, fine-tuning or from scratch — best accuracy for the compute spent?** All three are scored by the same harness, so the comparison is fair.
- **4. Do the models learn the rule, or memorise the pattern?** Tested on larger circuits, held-out language constructs, and real human-written code.

References

Papers

[1] Bahare Fatemi, Jonathan Halcrow and Bryan Perozzi. Talk like a Graph: Encoding Graphs for Large Language Models, arXiv preprint, 2023. https://arxiv.org/abs/2310.04560

[2] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve and Sida I. Wang. CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution, arXiv preprint, 2024. https://arxiv.org/abs/2401.03065

[3] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri and Juan Cruz-Benito. Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models, arXiv preprint, 2024. https://arxiv.org/abs/2406.14712

[4] Rui Yang, Ziruo Wang, Yuntian Gu, Tianyi Chen, Yitao Liang and Tongyang Li. QCircuitBench: A Large-Scale Dataset for Benchmarking Quantum Algorithm Design, arXiv preprint, 2024. https://arxiv.org/abs/2410.07961

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, arXiv preprint, 2021. https://arxiv.org/abs/2106.09685

Websites

[6] Grant Sanderson (3Blue1Brown). https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi. Accessed on: 20th September, 2026.

[7] Andrej Karpathy. https://www.youtube.com/watch?v=zjkBMFhNj_g. Accessed on: 20th September, 2026.

[8] Jay Alammar. https://jalammar.github.io/illustrated-transformer/. Accessed on: 20th September, 2026.

[9] IBM Qiskit. https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.dagcircuit.DAGCircuit. Accessed on: 20th September, 2026.

[10] Munich Quantum Toolkit. https://github.com/munich-quantum-toolkit/bench. Accessed on: 20th September, 2026.

[11] Pacific Northwest National Laboratory. https://github.com/pnnl/QASMBench. Accessed on: 20th September, 2026.